

SDA Past Papers - Notes and MCQs

Contents

1	System Concepts	7
1.1	Definition	7
1.2	System Types	7
2	Software Design	7
3	Object-Oriented Programming (OOP)	7
3.1	Definition	7
3.2	Basic Concepts	7
3.3	OOP Principles	7
3.4	Example: Abstraction	8
3.5	Example: Encapsulation	8
3.6	Example: Inheritance	8
3.7	Example: Polymorphism	9
3.8	Types of Polymorphism	9
3.8.1	Compile-Time Polymorphism (Static Polymorphism)	9
3.8.2	Run-Time Polymorphism (Dynamic Polymorphism)	10
4	Software Development Life Cycle (SDLC)	10
4.1	Phases of SDLC	10
4.2	Implementation Strategies	13
4.3	System Environments	13
4.4	Roles in SDLC	13
5	MCQs	13
6	Introduction	15
7	Software Process Models	15
7.1	Definition	15
7.2	Common Activities	15
7.3	Generic Process Flows	15
7.4	Waterfall Model	15
7.5	Incremental Model	15

7.6	Iteration Model and Rapid Software Development	16
7.7	Agile Characteristics	16
7.8	Plan-driven vs Agile Development	16
7.9	Agile Models Examples	16
8	Unified Process (UP)	16
8.1	UP Phases	16
8.2	UP Best Practices	17
8.3	UP Building Blocks	17
9	UML (Unified Modeling Language)	17
9.1	Domain Modeling	17
9.2	User Stories	17
9.3	Noun Phrase Identification	18
9.4	Associations and Multiplicity	18
10	MCQs	18
11	Introduction to Design Patterns	19
11.1	What is a Design Pattern?	19
11.2	Why Do We Need Design Patterns?	19
11.3	Definition of a Pattern	20
11.4	Patterns in Engineering	20
11.5	Gang of Four (GoF)	20
11.6	Evolution of Design Patterns	20
11.7	Types of Design Patterns	20
12	Multiple Choice Questions (MCQs)	21
13	Types of Design Patterns	22
13.1	Creational Design Patterns	22
13.2	List of Creational Patterns	23
13.3	Structural Design Patterns	23
13.4	List of Structural Patterns	24
13.5	Behavioral Design Patterns	24
13.6	List of Behavioral Patterns	25
14	Multiple Choice Questions (MCQs)	25
15	Elements of a Design Pattern	27
15.1	Four Elements of a Design Pattern	27
15.2	Pattern Name	27
15.3	Problem	28
15.4	Solution	28
15.5	Consequences	28

16 Multiple Choice Questions (MCQs)	29
17 Factory Method Design Pattern	30
17.1 Intent	31
17.2 Definition	31
17.3 Also Known As (AKA)	31
17.4 Applicability	31
17.5 Participants	32
17.5.1 Product	32
17.5.2 ConcreteProduct	32
17.5.3 Creator	32
17.5.4 ConcreteCreator	32
17.6 UML Class Diagram Description	32
18 Multiple Choice Questions (MCQs)	33
19 Structural Design Patterns	34
19.1 Key Characteristics	35
19.2 Purpose	35
20 Adapter Design Pattern	35
20.1 Intent	35
20.2 Problem	35
20.3 Motivation	35
20.4 Goal of Adapter Pattern	36
20.5 Object Model Adapter Pattern	36
20.6 Example: Media Player	36
21 Multiple Choice Questions (MCQs)	37
22 UML Class Diagram for Adapter Pattern	38
22.1 Description	39
23 Adapter Pattern Example	39
23.1 Target Code	39
23.2 Adaptee Code	39
23.3 Adapter Code	39
23.4 Client Code	39
24 Flow of Events in Adapter Pattern	40
25 Components of Adapter Pattern	40
25.1 Adaptee	40
25.2 Target	40
25.3 Adapter	40
25.4 Client	40

26 Consequences of Adapter Pattern	40
26.1 Class Adapter	40
26.2 Object Adapter	41
27 Class Adapter vs Object Adapter	41
27.1 Class Adapter	41
27.2 Object Adapter	41
28 Limitations of Adapter Pattern	41
29 When to Use Adapter Pattern	41
30 Multiple Choice Questions (MCQs)	42
31 State Chart Diagrams	43
31.1 Introduction	43
31.2 Classification of UML Diagrams	44
31.3 Why Use State Chart Diagrams?	44
31.4 Main Elements of State Chart Diagrams	44
31.5 States	45
31.6 State Compartments	45
31.7 Initial and Final States	45
31.8 Transitions	45
31.9 Events	46
31.10 Types of Events	46
31.11 Guard Conditions	46
31.12 Composite States	46
31.13 Concurrent States	46
31.14 Internal Transitions	47
31.15 Pseudo-States	47
31.16 Activity Diagram vs State Chart Diagram	47
32 MCQs: State Chart Diagrams	47
33 4+1 View Model	49
33.1 Introduction	49
33.2 What is a Model?	50
33.3 Why Do We Need Models?	50
33.4 Diagrams vs Models	50
33.5 Views	50
33.6 What is a View Model?	50
33.7 Intent of the 4+1 View Model	51
33.8 The Five Views of the 4+1 Model	51
33.8.1 Use Case View (The +1 View)	51
33.8.2 Logical View	51
33.8.3 Process View	51

33.8.4	Implementation View	51
33.8.5	Deployment View	52
33.9	Why is it Called 4+1 Instead of 5?	52
33.10	Homogenization of the System	52
33.11	Why Homogenize?	52
33.12	Elements of Homogenization	52
33.13	Combining Classes	53
33.14	Splitting Classes	53
33.15	Eliminating Classes	53
33.16	Consistency Checking	53
33.17	Scenario Walk-Through	53
33.18	Event Tracing	53
33.19	Documentation Review	53
34	MCQs: 4+1 View Model	54
35	MCQs: 4+1 View Model (Easy to Tricky)	55
36	MCQs: Implementation Diagrams	58
37	Deployment Diagrams	59
37.1	Introduction	59
37.2	Hardware Elements (Nodes)	60
37.3	Connections	60
37.4	Artifacts	60
37.5	When to Use Deployment Diagrams	60
37.6	Example Elements in a Deployment Diagram	60
37.7	MCQs: Deployment Diagrams	61
38	Timing Diagrams	62
38.1	Introduction	62
38.2	Elements of Timing Diagram	62
38.3	Timing Diagram Notations	62
38.4	Examples	62
38.5	4+1 View Model and Homogenization of Classes	63
38.6	MCQs	63
39	Entity, Control, and Boundary (ECB) Design Pattern	64
39.1	Overview	64
39.2	Boundary Objects	64
39.3	Control Objects	64
39.4	Entity Objects	64
39.5	ECB Communication Rules	64
39.6	Example: University Course Registration System	65
39.7	Robustness Diagram	65
39.8	MCQs: ECB Pattern	65

40 Use Case Realization and Analysis Classes	66
40.1 Use Case Realization	66
40.2 Responsibilities	67
40.3 Analysis Classes - Early Conceptual Model	67
40.4 Identifying Candidate Classes from Behavior	67
40.5 Analysis Classes: A First Step Towards Executables	67
40.6 Entity, Control, and Boundary (ECB) Design Pattern	67
40.7 Example: University Course Registration System	68
40.8 MCQs	68
40.9 MCQs on Use-Case Realization	69
41 Use Cases	70
41.1 Overview	70
41.2 Use Case Diagram Components	70
41.3 Types of Actors	70
41.4 Relationships in Use Cases	71
41.5 Types of Use Cases	71
41.6 Writing a Use Case	71
41.7 Example Use Case: Purchase at Checkout	71
41.8 Exercises	72
41.9 MCQs	72

1 System Concepts

1.1 Definition

- A system is an organized way of dealing with a problem.
- Systems are created to solve problems.
- Major components: **Input** → **Processing** → **Output**.

1.2 System Types

- **Application Program**: Performs coordinated tasks for the user.
- **Information System**: Helps organize and analyze data to solve organizational problems.

2 Software Design

- Goal: Build a model meeting customer requirements and enabling successful implementation.
- Activities: Identify objects, relationships, and produce design that can be converted to executables.
- Importance: Handles complexity, scale, and provides architecture guidance.

3 Object-Oriented Programming (OOP)

3.1 Definition

- OOP is a paradigm organizing code into objects containing both data and methods.
- Advantages: Modularity, reusability, flexibility, readability.

3.2 Basic Concepts

- **Objects**: Basic components performing a small piece of work.
- **Classes**: Blueprints for objects.

3.3 OOP Principles

1. **Abstraction**: Hiding implementation details and exposing only essential functionality to the user.
2. **Encapsulation**: Wrapping data and related methods into a single unit (class) and restricting direct access using access modifiers.

3. **Inheritance:** Reusing and extending existing code through *is-a* relationships between classes.
4. **Polymorphism:** Ability of a method or object to take multiple forms.

3.4 Example: Abstraction

```
abstract class Animal {  
    public abstract void animalSound();  
    public void sleep() {  
        System.out.println("Zzz");  
    }  
}
```

Explanation:

- The abstract method `animalSound()` hides implementation.
- Subclasses must provide their own implementation.

3.5 Example: Encapsulation

```
public class Person {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String newName) {  
        this.name = newName;  
    }  
}
```

Explanation:

- Data members are kept private.
- Access is controlled through public getter and setter methods.

3.6 Example: Inheritance

```
class Vehicle {  
    protected String brand = "Ford";  
}  
  
class Car extends Vehicle {  
    private String modelName = "Mustang";  
}
```


Explanation:

- Car inherits properties of Vehicle.
- Promotes code reuse and hierarchical relationships.

3.7 Example: Polymorphism

```
class Animal {
    public void animalSound() {
        System.out.println("Animal makes a sound");
    }
}

class Pig extends Animal {
    public void animalSound() {
        System.out.println("Pig says: wee wee");
    }
}

class Dog extends Animal {
    public void animalSound() {
        System.out.println("Dog says: bow wow");
    }
}
```

3.8 Types of Polymorphism

3.8.1 Compile-Time Polymorphism (Static Polymorphism): Compile-time polymorphism occurs when the method call is resolved at **compile time**. It is achieved using **method overloading** or **operator overloading**.

Key Features:

- Same method name with different parameter lists.
- Determined during compilation.
- Faster execution as binding occurs early.

Example: Method Overloading

```
class MathOperations {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

3.8.2 Run-Time Polymorphism (Dynamic Polymorphism): Run-time polymorphism occurs when the method call is resolved at **run time**. It is achieved using **method overriding** and inheritance.

Key Features:

- Same method signature in parent and child class.
- Decision is made at runtime based on object type.
- Supports dynamic method dispatch.

Example: Method Overriding

```
Animal myAnimal = new Dog();  
myAnimal.animalSound();    // Calls Dog's method
```

Explanation:

- Reference type is Animal.
- Object type is Dog.
- Correct method is selected at runtime.

4 Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured process used to design, develop, test, and maintain software systems. It ensures high-quality software that meets user requirements and is delivered within time and budget constraints.

4.1 Phases of SDLC

1. System Study

- Initial phase of SDLC.
- Understanding the existing system and identifying problems.
- Gathering preliminary requirements from users and stakeholders.
- Defines the scope and objectives of the system.

2. Feasibility Study

- Determines whether the proposed system is practical and viable.
- Evaluates different types of feasibility:
 - Technical feasibility
 - Economic feasibility
 - Operational feasibility

- Legal feasibility
 - Schedule feasibility
- Helps management decide whether to proceed with the project.

3. System Analysis

- Detailed study of user requirements.
- Functional and non-functional requirements are identified.
- Creation of requirement documents such as SRS (Software Requirement Specification).
- Use of tools like DFDs, use case diagrams, and ER diagrams.

4. System Design

- Converts requirements into a blueprint for implementation.
- Defines system architecture, modules, interfaces, and data structures.
- Includes:
 - High-Level Design (HLD)
 - Low-Level Design (LLD)
- UML diagrams are commonly used in this phase.

5. Coding

- Actual development of software begins.
- Developers write source code according to design specifications.
- Programming languages, frameworks, and tools are selected.
- Coding standards and best practices are followed.

6. Testing

- Verifies that the software works as expected.
- Detects and fixes defects or bugs.
- Includes different testing levels:
 - Unit Testing
 - Integration Testing
 - System Testing
 - Acceptance Testing
- Ensures software quality and reliability.

Testing Phase Details

Unit Testing: Unit testing is the first level of testing where **individual units or components** of the software are tested in isolation.

Key Points:

- Performed on individual functions, methods, or classes.
- Usually carried out by developers.
- Ensures that each unit works correctly according to design.
- Helps detect errors early in the development cycle.
- Uses test cases with known inputs and expected outputs.

Example:

- Testing a single function that calculates total price.
- Testing a method that validates user login.

Integration Testing: Integration testing is performed after unit testing to verify the **interaction between integrated modules**.

Key Points:

- Focuses on data flow and interface communication.
- Detects issues caused by module interaction.
- Ensures combined components work together correctly.

Types of Integration Testing:

- Top-Down Integration Testing
- Bottom-Up Integration Testing
- Big Bang Integration Testing
- Sandwich (Hybrid) Integration Testing

Example:

- Testing interaction between login module and database module.
- Testing communication between payment and order modules.

7. Implementation

- Deployment of the software into the real-world environment.
- User training and documentation are provided.
- Data migration may be performed.

- Can follow different strategies:
 - Direct implementation
 - Parallel implementation
 - Phased implementation

8. Maintenance

- Ongoing phase after deployment.
- Fixes bugs discovered after release.
- Improves performance and adapts software to new requirements.
- Includes:
 - Corrective maintenance
 - Adaptive maintenance
 - Perfective maintenance
 - Preventive maintenance

4.2 Implementation Strategies

- **Parallel Run:** Run manual and computerized systems together.
- **Pilot Run:** Install and test part of the system first.

4.3 System Environments

Development, Test, Staging, Pre-Production, Production, Mirror

4.4 Roles in SDLC

Developer, Development Manager, Project Manager, Test Manager, Configuration Manager, Deployment/Implementation Team

5 MCQs

1. What is a system? A) A random collection of programs B) **An organized way of dealing with a problem** C) Only hardware D) Only software
2. Which of the following is NOT a major component of a system? A) Input B) Processing C) Output D) **Memory storage**
3. Which of the following is a type of system? A) Application Program B) Information System C) Both A and B D) None of the above
4. What is the main goal of software design? A) To write code only B) **To build a model that meets customer requirements** C) To design the database only D) To train users

5. Which OOP principle is about hiding implementation details? A) Encapsulation B) Inheritance C) **Abstraction** D) Polymorphism
6. Which OOP principle allows objects to reuse code from other classes? A) Abstraction B) **Inheritance** C) Encapsulation D) Polymorphism
7. Which OOP principle restricts access to class variables and methods? A) Polymorphism B) **Encapsulation** C) Abstraction D) Inheritance
8. Which OOP principle allows methods to behave differently in different objects? A) Encapsulation B) Inheritance C) **Polymorphism** D) Abstraction
9. What is the first phase of SDLC? A) System Design B) **System Study** C) Coding D) Testing
10. Which SDLC phase evaluates economic and technical feasibility? A) System Study B) **Feasibility Study** C) Implementation D) Maintenance
11. Which SDLC phase converts design into programs? A) System Analysis B) **Coding** C) Testing D) Implementation
12. What type of test checks if all individual programs work together correctly? A) Unit Test B) **Integration Test** C) System Test D) Acceptance Test
13. Which system implementation strategy runs both manual and computerized systems together? A) **Parallel Run** B) Pilot Run C) Phased Implementation D) Direct Cutover
14. What is encapsulation in OOP? A) Dividing code into objects B) **Wrapping data and code together** C) Using multiple forms of a method D) Inheriting properties from parent classes
15. What is abstraction in OOP? A) **Hiding implementation details and showing functionality** B) Wrapping code and data together C) Creating multiple forms of methods D) Inheriting code from parent classes
16. Which OOP concept is demonstrated when `class Car extends Vehicle` in Java? A) Polymorphism B) **Inheritance** C) Encapsulation D) Abstraction
17. Which type of polymorphism occurs when the compiler cannot decide which method to call at compile time? A) Static / Compile-Time B) **Dynamic / Run-Time** C) Abstraction D) Encapsulation
18. What is the purpose of a model in software design? A) To show only the code B) **To simplify reality and help understand the system** C) To train users D) To implement the database
19. Which SDLC phase involves user training? A) Coding B) Testing C) **Implementation** D) System Analysis

20. Which SDLC phase is responsible for evaluating performance and making major changes if needed? A) **Maintenance** B) System Study C) Feasibility Study D) System Design

6 Introduction

- Software processes guide **who does what, when, and how** to achieve goals.
- Workers = Who, Activities = How, Artifacts = What, Workflows = When.

7 Software Process Models

7.1 Definition

- Process Model: A description of tasks, their sequence, conditions, and responsible actors to achieve results.
- Provides systematic coordination and control over tasks and personnel.

7.2 Common Activities

- Specification: What system should do.
- Design & Implementation: Organization and coding.
- Validation: Checks functionality.
- Evolution: Changes per customer needs.

7.3 Generic Process Flows

- Linear, Iterative, Incremental.

7.4 Waterfall Model

- Sequential stages: Analysis → Design → Code → Test → Maintenance.
- Pros: Simple, suitable for small projects.
- Cons: Inflexible, no working product until end.

7.5 Incremental Model

- Develop major requirements separately, integrate continuously.
- Pros: Fast, flexible, easier to test, customer feedback.
- Cons: Needs planning and clear system definition.

7.6 Iteration Model and Rapid Software Development

- Rapid delivery for fast-changing business requirements.
- Agile methods emerged to reduce delivery time and allow iterative evolution.

7.7 Agile Characteristics

- Interleaved specification, design, and implementation.
- Frequent version releases for evaluation.
- Minimal documentation, focus on working code.

7.8 Plan-driven vs Agile Development

- Plan-driven: Stages planned in advance, iteration within activities.
- Agile: Interleaved development, negotiation-based outputs.

7.9 Agile Models Examples

- XP (Extreme Programming)
- Scrum

8 Unified Process (UP)

- Iterative, incremental, risk-driven process for OO systems.
- Combines spiral and evolutionary models.
- Phases: Inception, Elaboration, Construction, Transition.
- Focuses on high-risk and high-value areas first.

8.1 UP Phases

- **Inception:** Vision, business case, scope, rough estimates. Inception is not a requirements phase.
- **Elaboration:** Core architecture, high-risk resolution, 80% requirements identified. Note: Elaboration is not the requirements or design phase; rather, it is a phase where the core architecture is iteratively implemented, and high risk issues are mitigated.
- **Construction:** Remaining development, lower-risk implementation. Iterative implementation of the remaining lower risk and easier elements, and preparation for deployment. Note: Construction is not the implementation phase
- **Transition:** Beta testing, deployment.

8.2 UP Best Practices

- Early risk handling.
- Continuous user feedback.
- Test early and often.
- Use cases and visual modeling with UML.

8.3 UP Building Blocks

- Workers = Who performs activities.
- Activities = How work is done.
- Artifacts = What is produced.
- Workflows = When activities occur.

9 UML (Unified Modeling Language)

- Language to specify, visualize, construct, and document software and business artifacts.
- Perspectives: Conceptual, Specification, Implementation.
- UML as Sketch, Blueprint, or Programming Language.

9.1 Domain Modeling

- Domain Model: Conceptual objects in problem domain.
- Characteristics: Visual, attributes, associations, no operations/functions.
- Steps:
 1. Create User Stories
 2. Identify candidate conceptual classes
 3. Draw UML class diagram
 4. Add associations and multiplicity
 5. Add attributes
 6. Use domain vocabulary

9.2 User Stories

- Short sentences describing **who**, **what**, **why**.
- Captures problem definition, scope, and vision.

9.3 Noun Phrase Identification

- Identify candidate classes (nouns/noun phrases).
- Determine if noun is a class (state/behaviors) or attribute (primitive value).
- Refine objects, exclude irrelevant items.

9.4 Associations and Multiplicity

- Identify meaningful connections between classes.
- Multiplicity: How many instances of a class relate to another class.
- Avoid redundant associations.
- Complex objects → association; primitive values → attribute.

10 MCQs

1. Which of the following best defines a software process? A) Coding only B) **A set of activities defining who does what, when, and how** C) Testing only D) Documentation only
2. Which is NOT a common activity in a process model? A) Specification B) Design and Implementation C) Validation D) **Marketing**
3. What is the main disadvantage of the Waterfall model? A) Easy to understand B) Works for small projects C) **Inflexible, no working product until the end** D) Sequential process
4. Incremental model is best suited for: A) Well-understood requirements only B) **Projects needing early working software and flexibility** C) Projects with single release D) None
5. Agile development emphasizes: A) Extensive documentation B) **Frequent delivery, interleaved design and coding, user involvement** C) No testing D) Waterfall approach
6. Unified Process (UP) is: A) Waterfall model B) **Iterative, incremental, risk-driven for OO systems** C) Agile XP only D) Scrum only
7. Which UP phase identifies high-risk areas and core architecture? A) Inception B) **Elaboration** C) Construction D) Transition
8. In domain modeling, operations/functions are: A) Included in domain model B) **Not included, assigned during design** C) Optional D) Mandatory
9. In UML, a conceptual class is: A) A software artifact B) **A representation of a domain concept, not software class** C) A database table D) A method

10. Multiplicity in UML indicates: A) Number of attributes in a class B) Number of methods in a class C) **Number of instances of a class that can be associated with another instance** D) Number of users
11. A complex object (e.g., Airport) in a domain model should be: A) Attribute of another class B) **Associated with another class via association** C) Ignored D) Represented as primitive type
12. What is the purpose of user stories? A) Capture programming details B) **Capture who, what, why of requirements in simple terms** C) Replace UML D) Test software
13. Noun phrase identification helps in: A) Determining UML relationships B) Identifying candidate classes and attributes C) Writing code D) Deployment
14. Which is NOT a UP best practice? A) Test early and often B) Constant user feedback C) Early core architecture D) **Focus on documentation over software product**
15. Which agile model is focused on sprints and iterative feature delivery? A) Waterfall B) XP C) **Scrum** D) UP

11 Introduction to Design Patterns

11.1 What is a Design Pattern?

A **design pattern** is a general, reusable solution to a commonly occurring problem in software design. It is **not a finished design** that can be directly converted into code. Instead, it is a **template or guideline** for solving a problem that can be applied in many situations.

Key Points:

- Design patterns describe proven solutions.
- They improve design quality and development speed.
- They promote reuse of software architecture.
- They help designers reach correct designs faster.

11.2 Why Do We Need Design Patterns?

- Avoid reinventing solutions to common problems.
- Improve code flexibility and maintainability.
- Facilitate communication among developers using standard terminology.
- Reduce software development cost and time.

11.3 Definition of a Pattern

A **pattern** is a recurring solution to a standard problem within a specific context.

Christopher Alexander’s Definition:

“A pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”

11.4 Patterns in Engineering

- Mature engineering disciplines use handbooks of proven solutions.
- Automobile engineers reuse standard designs instead of starting from physics laws.
- Software engineering also benefits from reusable design solutions.

11.5 Gang of Four (GoF)

The term **Gang of Four** refers to:

- Erich Gamma
- Richard Helm
- Ralph Johnson
- John Vlissides

Their book *Design Patterns: Elements of Reusable Object-Oriented Software* (1995) catalogues **23 design patterns** in C++ and Smalltalk.

11.6 Evolution of Design Patterns

- First part of GoF book discusses OOP principles.
- Second part presents 23 reusable design patterns.
- Patterns evolved from real-world software development experience.

11.7 Types of Design Patterns

There are three main categories:

1. **Creational Patterns** – Concerned with object creation.
2. **Structural Patterns** – Deal with class and object composition.
3. **Behavioral Patterns** – Focus on interaction and responsibility among objects.

12 Multiple Choice Questions (MCQs)

1. A design pattern is:
 - (a) A complete software program
 - (b) A reusable code library
 - (c) A general reusable solution to a common design problem
 - (d) A programming language

Answer: (c)

2. Design patterns primarily help in:
 - (a) Increasing code size
 - (b) Reducing runtime errors only
 - (c) Achieving better software design faster
 - (d) Eliminating documentation

Answer: (c)

3. Who introduced the concept of patterns originally?
 - (a) Erich Gamma
 - (b) Christopher Alexander
 - (c) John Vlissides
 - (d) Ralph Johnson

Answer: (b)

4. The GoF book contains how many design patterns?
 - (a) 12
 - (b) 15
 - (c) 23
 - (d) 30

Answer: (c)

5. Which of the following is NOT a type of design pattern?
 - (a) Creational
 - (b) Structural
 - (c) Behavioral
 - (d) Functional

Answer: (d)

6. Creational patterns mainly focus on:

- (a) Object communication
- (b) Object creation mechanisms
- (c) System performance
- (d) Data storage

Answer: (b)

7. Structural patterns deal with:

- (a) Algorithms
- (b) Object creation
- (c) Class and object composition
- (d) Memory management

Answer: (c)

8. Behavioral patterns emphasize:

- (a) Object inheritance
- (b) Interaction between objects
- (c) Database design
- (d) Compilation process

Answer: (b)

13 Types of Design Patterns

13.1 Creational Design Patterns

Creational design patterns deal with **object creation and initialization** while hiding the creation logic. They provide flexibility by deciding **which objects should be created** for a given situation.

Key Features:

- Encapsulate object creation logic.
- Reduce tight coupling between client code and concrete classes.
- Improve flexibility and reuse.

Examples of Creational Patterns:

- Singleton
- Factory
- Factory Method
- Abstract Factory
- Builder
- Prototype

13.2 List of Creational Patterns

- Singleton
- Factory
- Factory Method
- Abstract Factory
- Builder
- Prototype

13.3 Structural Design Patterns

Structural design patterns focus on **how classes and objects are combined** to form larger structures. They deal with **relationships between entities** and make systems easier to extend and maintain.

Key Features:

- Concerned with class and object composition.
- Use inheritance to compose interfaces.
- Define ways to combine objects to obtain new functionality.

Examples of Structural Patterns:

- Adapter
- Bridge
- Facade
- Decorator
- Proxy
- Flyweight

13.4 List of Structural Patterns

- Adapter
- Bridge
- Facade
- Decorator
- Proxy
- Flyweight

13.5 Behavioral Design Patterns

Behavioral design patterns focus on **communication and interaction between objects**. They make it easier and more flexible for entities to work together.

Key Features:

- Concerned with object interaction.
- Define responsibility and communication patterns.
- Reduce complex conditional logic.

Examples of Behavioral Patterns:

- Chain of Responsibility
- Command
- Interpreter
- Observer
- Strategy
- State
- Iterator
- Visitor
- Template Method

13.6 List of Behavioral Patterns

- Chain of Responsibility
- Command
- Interpreter
- Observer
- Strategy
- State
- Iterator
- Visitor
- Template Method

14 Multiple Choice Questions (MCQs)

1. Creational design patterns mainly deal with:

- (a) Object communication
- (b) Object creation and initialization
- (c) Object destruction
- (d) Data storage

Answer: (b)

2. Which pattern ensures only one instance of a class exists?

- (a) Factory
- (b) Builder
- (c) Singleton
- (d) Prototype

Answer: (c)

3. Structural patterns focus on:

- (a) Object creation
- (b) Algorithm implementation
- (c) Class and object composition
- (d) Program execution speed

Answer: (c)

4. Which of the following is a structural pattern?

- (a) Observer
- (b) Adapter
- (c) Strategy
- (d) Command

Answer: (b)

5. Behavioral patterns are mainly concerned with:

- (a) Object creation
- (b) Class inheritance
- (c) Communication between objects
- (d) Memory management

Answer: (c)

6. Which of the following is NOT a creational pattern?

- (a) Factory
- (b) Singleton
- (c) Adapter
- (d) Builder

Answer: (c)

7. Chain of Responsibility belongs to which category?

- (a) Creational
- (b) Structural
- (c) Behavioral
- (d) Architectural

Answer: (c)

8. Which pattern allows incompatible interfaces to work together?

- (a) Facade
- (b) Adapter
- (c) Proxy
- (d) Flyweight

Answer: (b)

9. Behavioral patterns help to:

- (a) Reduce coupling
- (b) Increase object creation
- (c) Simplify object communication
- (d) Replace inheritance

Answer: (c)

15 Elements of a Design Pattern

A design pattern is described using four essential elements. These elements help programmers understand, communicate, and evaluate design patterns effectively.

15.1 Four Elements of a Design Pattern

The four main elements are:

1. Pattern Name
2. Problem
3. Solution
4. Consequences

15.2 Pattern Name

The **pattern name** is a one or two word description that identifies the pattern.

Key Points:

- Provides a common vocabulary for designers and programmers.
- Enables communication at a higher level of abstraction.
- Helps designers think about designs and trade-offs.
- The name should recall both the problem and the solution.

Examples of Pattern Names:

- Factory Method
- Singleton
- Mediator
- Prototype

15.3 Problem

The **problem** element describes:

- When the pattern should be applied.
- The general intent of the pattern.
- The specific design issue or motivation.

Example (Singleton Pattern):

- **Intent:** Ensure that only one instance of a class exists.
- **Motivation:** Prevent multiple objects from accessing a system's audio hardware at the same time.

15.4 Solution

The **solution** describes the elements that make up the pattern.

Includes:

- Classes and objects involved.
- Methods, interfaces, and data structures.
- Relationships and collaborations between elements.
- Responsibilities of each component.

Important Note:

- The structure and relationships are more important than individual components.
- Components can change without altering the core pattern.

15.5 Consequences

The **consequences** describe the results and trade-offs of applying a pattern.

Key Points:

- Multiple patterns may solve the same problem.
- The final choice often depends on consequences.
- Some patterns require more memory.
- Some patterns may be slower or faster.
- Some patterns are more scalable than others.

Consequences help in:

- Evaluating trade-offs.
- Understanding performance and flexibility.
- Selecting the most appropriate pattern.

16 Multiple Choice Questions (MCQs)

1. How many elements are used to describe a design pattern?

- (a) Two
- (b) Three
- (c) Four
- (d) Five

Answer: (c)

2. Which element provides a common vocabulary among programmers?

- (a) Problem
- (b) Solution
- (c) Pattern Name
- (d) Consequences

Answer: (c)

3. The problem element of a design pattern describes:

- (a) Code implementation
- (b) When and why the pattern should be used
- (c) Memory usage
- (d) Programming language syntax

Answer: (b)

4. The intent of the Singleton pattern is to:

- (a) Create multiple objects
- (b) Improve performance
- (c) Ensure only one instance of a class exists
- (d) Simplify inheritance

Answer: (c)

5. Which element explains classes, methods, and their relationships?

- (a) Pattern Name
- (b) Problem
- (c) Solution
- (d) Consequences

Answer: (c)

6. In design patterns, which is considered more important than individual components?

- (a) Variables
- (b) Syntax
- (c) Relationships and structure
- (d) Programming language

Answer: (c)

7. Consequences of a pattern mainly describe:

- (a) Source code
- (b) Trade-offs and results
- (c) Naming conventions
- (d) Compilation errors

Answer: (b)

8. Which factor often determines the final choice of a design pattern?

- (a) Pattern name
- (b) Programming language
- (c) Consequences
- (d) Number of classes

Answer: (c)

9. Some design patterns may differ in:

- (a) Memory usage and scalability
- (b) Compiler version
- (c) Keyboard input
- (d) Screen resolution

Answer: (a)

17 Factory Method Design Pattern

Type: Creational Design Pattern

17.1 Intent

The intent of the **Factory Method Pattern** is:

“Define an interface for creating an object, but let subclasses decide which class to instantiate.”

Key Points:

- Defers object instantiation to subclasses at run time.
- Refers to the newly created object through a common interface.
- Allows selecting one object from a set of related classes based on logic.
- Promotes loose coupling between client code and concrete classes.

17.2 Definition

The Factory Method Pattern:

- Defines an interface for creating objects.
- Allows subclasses to decide which concrete class to instantiate.
- Is one of the most widely used design patterns in software engineering.

17.3 Also Known As (AKA)

Virtual Constructor

Explanation:

- In C++, a virtual constructor allows creating objects without knowing their concrete type.
- Factory Method achieves this by delegating object creation to subclasses.
- Hence, Factory Method is referred to as a virtual constructor.

17.4 Applicability

Use the Factory Method pattern when:

- A class cannot anticipate the class of objects it must create.
- The application knows only the superclass (often abstract), not the concrete subclass.
- A class wants its subclasses to specify the objects it creates.
- Classes delegate responsibility to one of several helper subclasses.
- You want to localize knowledge of which subclass is responsible for object creation.

17.5 Participants

The Factory Method pattern consists of the following participants:

17.5.1 Product:

- Defines the interface of objects that the factory method creates.

17.5.2 ConcreteProduct:

- Implements the Product interface.
- Represents a specific object created by the factory.

17.5.3 Creator:

- Declares the factory method.
- Returns an object of type Product.
- May provide a default implementation of the factory method.
- Relies on subclasses to decide which ConcreteProduct to instantiate.

17.5.4 ConcreteCreator:

- Overrides the factory method.
- Returns an instance of a ConcreteProduct.

17.6 UML Class Diagram Description

The UML class diagram of the Factory Method pattern includes:

- An abstract **Product** interface.
- One or more **ConcreteProduct** classes implementing Product.
- An abstract **Creator** class declaring the factory method.
- One or more **ConcreteCreator** classes overriding the factory method.

Key Relationship:

- Creator depends on Product, not on ConcreteProduct.
- ConcreteCreator decides which ConcreteProduct to instantiate.

18 Multiple Choice Questions (MCQs)

1. Factory Method pattern belongs to which category?

- (a) Structural
- (b) Behavioral
- (c) Creational
- (d) Architectural

Answer: (c)

2. The main intent of Factory Method pattern is to:

- (a) Create multiple objects
- (b) Define an interface and let subclasses decide instantiation
- (c) Improve system performance
- (d) Reduce memory usage

Answer: (b)

3. Factory Method pattern is also known as:

- (a) Adapter
- (b) Builder
- (c) Virtual Constructor
- (d) Prototype

Answer: (c)

4. Which participant defines the interface of objects created?

- (a) Creator
- (b) ConcreteCreator
- (c) Product
- (d) ConcreteProduct

Answer: (c)

5. Which class overrides the factory method?

- (a) Product
- (b) Creator
- (c) ConcreteProduct
- (d) ConcreteCreator

Answer: (d)

6. Factory Method helps in:

- (a) Tight coupling
- (b) Reducing flexibility
- (c) Deferring object creation
- (d) Increasing compile-time dependency

Answer: (c)

7. When is Factory Method pattern useful?

- (a) When object types are fixed
- (b) When subclasses decide object creation
- (c) When memory usage must be minimized
- (d) When only one object is required

Answer: (b)

8. Creator class mainly depends on:

- (a) ConcreteProduct
- (b) ConcreteCreator
- (c) Product interface
- (d) Implementation details

Answer: (c)

9. Factory Method is most widely used because it:

- (a) Eliminates inheritance
- (b) Supports loose coupling
- (c) Avoids abstraction
- (d) Replaces all other patterns

Answer: (b)

19 Structural Design Patterns

Structural design patterns deal with **how classes and objects are combined** to form larger structures.

19.1 Key Characteristics

- Focus on class and object composition.
- Use inheritance to compose interfaces or implementations.
- Identify relationships between entities.
- Simplify system design by organizing object structures.

19.2 Purpose

Structural patterns make systems easier to understand, extend, and maintain by clearly defining how different entities are connected.

20 Adapter Design Pattern

Category: Structural Design Pattern

20.1 Intent

The intent of the **Adapter Pattern** is:

- To convert the interface of an existing class into another interface that clients expect.
- To allow incompatible interfaces to work together.
- To wrap an existing class with a new interface.

Also Known As: Wrapper Pattern

20.2 Problem

- An existing or off-the-shelf component provides useful functionality.
- The interface of this component is incompatible with the system being developed.
- Client code expects a different interface.
- The source code of the existing component may not be available or should not be modified.

20.3 Motivation

- A toolkit or library cannot be used directly due to interface mismatch.
- The library may only be available as a compiled file (e.g., .class or .dll).
- Changing the library interface is not feasible or desirable.
- The adapter makes the existing code usable without modifying client code.

20.4 Goal of Adapter Pattern

- Keep client code unchanged.
- Introduce a new adapter class.
- Convert services of the existing class into a form expected by the client.
- Preserve original functionality while improving reusability.

20.5 Object Model Adapter Pattern

- Uses object composition rather than inheritance.
- The adapter contains an instance of the adaptee.
- Most commonly used form of adapter pattern.

20.6 Example: Media Player

Scenario:

- **MediaPlayer** is an interface.
- **AudioPlayer** implements MediaPlayer.
- AudioPlayer can play **mp3** files by default.

Extended Requirement:

- Support **v1c** and **mp4** formats.
- **AdvancedMediaPlayer** interface exists.
- Concrete classes implement AdvancedMediaPlayer for **v1c** and **mp4**.

Adapter Solution:

- **MediaAdapter** implements MediaPlayer.
- MediaAdapter uses AdvancedMediaPlayer internally.
- AudioPlayer uses MediaAdapter without knowing the concrete implementation.

Result:

- AudioPlayer can play multiple formats.
- Client code remains unchanged.
- System becomes extensible and reusable.

21 Multiple Choice Questions (MCQs)

1. Structural design patterns mainly focus on:

- (a) Object creation
- (b) Object communication
- (c) Class and object composition
- (d) Algorithm optimization

Answer: (c)

2. Adapter pattern belongs to which category?

- (a) Creational
- (b) Structural
- (c) Behavioral
- (d) Architectural

Answer: (b)

3. The main intent of Adapter pattern is to:

- (a) Improve performance
- (b) Convert one interface into another
- (c) Create multiple objects
- (d) Reduce memory usage

Answer: (b)

4. Adapter pattern is also known as:

- (a) Bridge
- (b) Wrapper
- (c) Facade
- (d) Proxy

Answer: (b)

5. Adapter pattern is useful when:

- (a) Interfaces are compatible
- (b) Source code is available
- (c) Interfaces are incompatible
- (d) Performance is critical

Answer: (c)

6. Which technique is commonly used in object adapter?

- (a) Multiple inheritance
- (b) Object composition
- (c) Polymorphism only
- (d) Static binding

Answer: (b)

7. In the Media Player example, which class acts as the adapter?

- (a) AudioPlayer
- (b) MediaPlayer
- (c) AdvancedMediaPlayer
- (d) MediaAdapter

Answer: (d)

8. Adapter pattern helps to:

- (a) Modify client code
- (b) Change existing library
- (c) Reuse existing code
- (d) Eliminate interfaces

Answer: (c)

9. Which of the following is NOT a reason to use Adapter pattern?

- (a) Interface mismatch
- (b) Library reuse
- (c) Object creation control
- (d) Client code preservation

Answer: (c)

22 UML Class Diagram for Adapter Pattern

The Adapter pattern UML consists of four main participants:

- **Client**
- **Target**
- **Adapter**
- **Adaptee**

22.1 Description

- The **Client** depends only on the **Target** interface.
- The **Adapter** implements the Target interface.
- The Adapter holds a reference to the **Adaptee**.
- Requests from the client are forwarded to the adaptee.

23 Adapter Pattern Example

23.1 Target Code

```
class Target {  
    public void request() {}  
}
```

23.2 Adaptee Code

```
class Adaptee {  
    public void specificRequest() {  
        System.out.println("Adaptee: SpecificRequest");  
    }  
}
```

23.3 Adapter Code

```
class Adapter extends Target {  
    private Adaptee adaptee;  
  
    public Adapter(Adaptee a) {  
        adaptee = a;  
    }  
  
    public void request() {  
        adaptee.specificRequest();  
    }  
}
```

23.4 Client Code

```
Adaptee a = new Adaptee();  
Target t = new Adapter(a);  
  
public void test() {  
    t.request();  
}
```

24 Flow of Events in Adapter Pattern

- Client calls a method on the Adapter using the Target interface.
- Adapter translates the request into a call to the Adaptee.
- Adaptee performs the requested operation.
- Client receives the result without knowing the adapter exists.

25 Components of Adapter Pattern

25.1 Adaptee

- Defines an existing interface that needs adapting.
- Represents the component with useful functionality.

25.2 Target

- Defines the domain-specific interface expected by the client.
- Client interacts only with this interface.

25.3 Adapter

- Implements the Target interface.
- Connects the Client with the Adaptee.
- Translates client requests into adaptee-specific calls.

25.4 Client

- Uses the Target interface.
- Is unaware of the Adaptee and Adapter implementation.

26 Consequences of Adapter Pattern

26.1 Class Adapter

- Uses inheritance to adapt Adaptee to Target.
- Allows overriding Adaptee behavior.
- Introduces no additional pointer indirection.
- Tightly coupled to a specific Adaptee class.

26.2 Object Adapter

- Uses composition instead of inheritance.
- Can work with multiple adaptees and subclasses.
- Can add functionality to all adaptees at once.
- Harder to override adaptee behavior.

27 Class Adapter vs Object Adapter

27.1 Class Adapter

- Uses inheritance.
- Can only wrap a class not an interface.
- Cannot adapt interfaces.
- Requires single inheritance.

27.2 Object Adapter

- Uses composition.
- Can wrap classes or interfaces.
- More flexible than class adapter.
- Preferred in most real-world scenarios.

28 Limitations of Adapter Pattern

- Introduces additional classes.
- Increases system complexity.
- Overuse can make design harder to understand.

29 When to Use Adapter Pattern

- When using third-party libraries with incompatible interfaces.
- When source code of existing class cannot be modified.
- When extending existing functionality without changing original code.
- When working with sealed classes or requiring multiple inheritance.

- Use **class adapter** for single inheritance needs.
- Use **object adapter** for flexible and reusable designs.

30 Multiple Choice Questions (MCQs)

1. Adapter pattern belongs to which category?

- (a) Creational
- (b) Structural
- (c) Behavioral
- (d) Architectural

Answer: (b)

2. The Adapter pattern is also known as:

- (a) Bridge Pattern
- (b) Proxy Pattern
- (c) Wrapper Pattern
- (d) Facade Pattern

Answer: (c)

3. Which component defines the interface expected by the client?

- (a) Adapter
- (b) Adaptee
- (c) Target
- (d) Client

Answer: (c)

4. In Adapter pattern, the client interacts with:

- (a) Adaptee directly
- (b) Adapter through Target interface
- (c) Concrete implementation
- (d) Library code

Answer: (b)

5. Object Adapter uses which technique?

- (a) Inheritance

- (b) Aggregation
- (c) Composition
- (d) Polymorphism only

Answer: (c)

6. Which adapter is more flexible?

- (a) Class Adapter
- (b) Object Adapter
- (c) Singleton Adapter
- (d) Factory Adapter

Answer: (b)

7. Adapter pattern is useful when:

- (a) Interfaces are compatible
- (b) Object creation must be controlled
- (c) Interfaces are incompatible
- (d) Only one object is needed

Answer: (c)

8. Which is a limitation of Adapter pattern?

- (a) Improves reusability
- (b) Adds extra classes
- (c) Keeps client unchanged
- (d) Supports legacy code

Answer: (b)

31 State Chart Diagrams

31.1 Introduction

A State Chart Diagram (also called a State Machine Diagram) is a behavioral UML diagram that models the dynamic behavior of an object throughout its life cycle. It represents how an object changes its state in response to external or internal events.

Unlike other dynamic diagrams that focus on interactions among multiple objects, a state chart diagram focuses on **one object at a time** and describes how that object reacts differently to the same event depending on its current state.

State chart diagrams are especially useful for modeling reactive and event-driven systems such as ATMs, traffic lights, elevators, order processing systems, and communication protocols.

31.2 Classification of UML Diagrams

UML diagrams are broadly classified into three categories based on the aspect of the system they model:

- **Static Diagrams** These diagrams describe the static structure of a system and represent what the system *is*.
 - **Use Case Diagram:** Shows system functionality from a user's perspective.
 - **Class Diagram:** Represents classes, attributes, methods, and relationships.
- **Dynamic Diagrams** These diagrams model the dynamic behavior of the system and represent how the system *behaves over time*.
 - **Activity Diagram:** Shows flow of control or workflow.
 - **Sequence Diagram:** Shows interaction between objects in time order.
 - **Object Diagram:** Snapshot of objects at a particular moment.
 - **State Diagram:** Shows state changes of an object.
 - **Collaboration Diagram:** Focuses on object relationships during interaction.
- **Implementation Diagrams** These diagrams focus on the physical implementation of the system.
 - **Component Diagram:** Shows software components and dependencies.
 - **Deployment Diagram:** Shows physical deployment of hardware and software.

31.3 Why Use State Chart Diagrams?

State chart diagrams are used for the following reasons:

- To model **state-dependent behavior** of an object.
- The same event may cause different actions depending on the current state.
- Helps in understanding the complete life cycle of an object.
- Useful for identifying missing states or transitions.
- Commonly used together with sequence diagrams for complete behavioral modeling.
- Best suited for reactive systems like ATMs, elevators, traffic lights, vending machines.

31.4 Main Elements of State Chart Diagrams

A state chart diagram consists of three fundamental elements:

- **States:** Conditions in which an object exists.
- **Transitions:** Movement from one state to another.
- **Events:** Occurrences that trigger transitions.

31.5 States

A state represents a condition or situation in the life of an object during which it performs some activity or waits for an event to occur.

An object always exists in exactly one state at any given time.

Examples:

- Invoice is *Paid*
- Engine is *Running*
- Order is *Pending*

31.6 State Compartments

A state may be divided into up to three compartments:

1. **Name Compartment** Displays the name of the state, such as Idle, Active, or Closed.
2. **Activity Compartment** Specifies actions associated with the state.
 - **entry:** Action performed when entering the state.
 - **exit:** Action performed when leaving the state.
 - **do:** Activity performed while staying in the state.
3. **Internal Transition Compartment** Lists transitions that occur without leaving the state.

31.7 Initial and Final States

- **Initial State:** Represents the starting point of the object. Shown by a filled black circle.
- **Final State:** Represents the termination of the object's life cycle. Shown by a filled circle inside another circle.

31.8 Transitions

Transitions define how an object moves from one state to another.

- Represented by arrows.
- Can be triggered by events.
- Labeled as: *event [guard] / action*
- A state may have a transition to itself (self-transition).

31.9 Events

An event is a significant occurrence that triggers a transition from one state to another.

Examples:

- Open
- Deposit(amount)
- Withdraw(amount)
- Close

31.10 Types of Events

UML defines four types of events:

- **Condition Event:** Triggered when a condition becomes true.
- **Signal Event:** Triggered by receiving a signal or message.
- **Call Event:** Triggered by invoking an operation.
- **Time Event:** Triggered after a specific time period.

31.11 Guard Conditions

A guard condition is a Boolean expression written inside square brackets and must evaluate to true for the transition to occur.

Examples:

balance $\geq$ amount

t = 15 sec

31.12 Composite States

A composite state is a state that contains other substates.

- **Sequential (Disjoint) Substates:** Only one substate is active at a time.
- **Concurrent (Orthogonal) Substates:** Multiple substates execute in parallel.

31.13 Concurrent States

Concurrent states allow parallel execution within a state.

- Represented using orthogonal regions.
- Fork pseudo-state splits execution.
- Join pseudo-state synchronizes execution.

31.14 Internal Transitions

Internal transitions occur when an event triggers an action but does not change the state.

- No state exit or entry occurs.
- Example: Typing on a keyboard without changing Caps Lock state.

31.15 Pseudo-States

Pseudo-states are temporary states used to control transitions.

- Choice pseudo-state (diamond)
- Junction pseudo-state
- Entry point
- Exit point
- History state (remembers previous state)
- Terminate state (cross symbol)

31.16 Activity Diagram vs State Chart Diagram

- Activity diagrams focus on the flow of activities.
- State chart diagrams focus on state changes of an object.
- Activity diagrams emphasize workflow, while state charts emphasize object life cycle.
- An activity diagram is a special case of a state chart diagram.

32 MCQs: State Chart Diagrams

1. State chart diagrams are mainly used to model:

- (a) Structure of a system
- (b) Data flow
- (c) Behavior of a single object
- (d) Deployment architecture

Answer: (c)

2. Which UML diagram shows state-dependent behavior?

- (a) Class Diagram
- (b) Sequence Diagram

- (c) State Chart Diagram
- (d) Component Diagram

Answer: (c)

3. Initial state in a state diagram is represented by:

- (a) Hollow circle
- (b) Filled circle
- (c) Rectangle
- (d) Diamond

Answer: (b)

4. Which event does NOT change the state?

- (a) External event
- (b) Time event
- (c) Internal transition
- (d) Signal event

Answer: (c)

5. Guard conditions are:

- (a) Activities
- (b) Boolean expressions
- (c) States
- (d) Messages

Answer: (b)

6. Composite states are used to:

- (a) Reduce transitions
- (b) Represent nested states
- (c) Show objects
- (d) Show activities

Answer: (b)

7. Concurrent states represent:

- (a) Sequential execution
- (b) Parallel execution

- (c) Conditional execution
- (d) Optional execution

Answer: (b)

8. Fork and Join pseudo-states are used in:

- (a) Activity diagrams
- (b) Class diagrams
- (c) Concurrent state machines
- (d) Use case diagrams

Answer: (c)

9. A terminate pseudo-state is represented by:

- (a) Circle
- (b) Diamond
- (c) Cross
- (d) Rectangle

Answer: (c)

10. Activity diagram differs from state chart diagram because:

- (a) Activity focuses on states
- (b) State chart focuses on activities
- (c) Activity focuses on control flow
- (d) State chart focuses on objects collaboration

Answer: (c)

33 4+1 View Model

33.1 Introduction

The **4+1 View Model** is a software architecture model proposed by Philippe Kruchten. It is used to describe the architecture of complex software systems using multiple, interrelated views. Each view addresses the concerns of different stakeholders such as developers, system administrators, testers, project managers, and end users.

The model consists of four main architectural views and one central view, hence the name **4+1**.

33.2 What is a Model?

A model is an abstraction that represents selected aspects of a system.

- Models help understand complex systems.
- A model focuses on important details and ignores irrelevant ones.
- Models never fully represent reality.
- Examples include architectural models, business models, and simulation models.

33.3 Why Do We Need Models?

- Facilitate communication among stakeholders.
- Visualize system structure and behavior.
- Verify completeness, correctness, and consistency.
- Help document decisions for future reference.

33.4 Diagrams vs Models

- A **diagram** represents a specific aspect of a system.
- A **model** provides a complete system view from a certain perspective.
- A model may consist of multiple diagrams and documentation.

33.5 Views

A view is a projection of the system that focuses on a specific concern.

- Reduces complexity by isolating concerns.
- Different views are interconnected.
- Each view is intended for specific stakeholders.

33.6 What is a View Model?

A view model defines a set of views used to describe the system architecture.

- Each view represents the system from a specific viewpoint.
- Helps manage complexity in large distributed systems.

33.7 Intent of the 4+1 View Model

- Separate system concerns into different views.
- Address needs of different stakeholders.
- Improve understanding, communication, and validation.

33.8 The Five Views of the 4+1 Model

33.8.1 Use Case View (The +1 View):

- Describes system functionality from the user's perspective.
- Captures functional requirements.
- Uses use case diagrams and activity diagrams.
- Central view that drives all other views.

33.8.2 Logical View:

- Describes how system functionality is provided.
- Focuses on classes, objects, and relationships.
- Intended for designers and developers.
- Uses class diagrams, object diagrams, and interaction diagrams.

33.8.3 Process View:

- Addresses concurrency, synchronization, and performance.
- Shows processes, threads, and communication.
- Important for scalability and reliability.
- Uses activity diagrams, sequence diagrams, and state diagrams.

33.8.4 Implementation View:

- Describes system organization into modules and components.
- Focuses on source code structure.
- Intended for developers.
- Uses component diagrams.

33.8.5 Deployment View:

- Describes physical deployment of software on hardware.
- Shows nodes and network connections.
- Intended for system administrators and testers.
- Uses deployment diagrams.

33.9 Why is it Called 4+1 Instead of 5?

- Use case view is central and drives other views.
- Other views exist to realize use case requirements.
- Once other views are complete, use case view becomes redundant.

33.10 Homogenization of the System

Homogenization is the process of making the system model consistent and uniform.

- Removes redundancy.
- Resolves naming conflicts.
- Ensures consistent interpretation across teams.

33.11 Why Homogenize?

- Multiple teams may use different terminology.
- Prevents inconsistencies in large projects.
- Continuous activity throughout the lifecycle.

33.12 Elements of Homogenization

- Combining Classes
- Splitting Classes
- Eliminating Classes
- Consistency Checking
- Scenario Walk-Through
- Event Tracing
- Documentation Review

33.13 Combining Classes

- Resolve class name conflicts.
- Merge classes with similar responsibilities.
- Choose names aligned with customer terminology.

33.14 Splitting Classes

- Classes should follow single responsibility principle.
- Separate unrelated responsibilities into distinct classes.

33.15 Eliminating Classes

- Remove classes with no behavior or structure.
- Remove control classes that only pass data.

33.16 Consistency Checking

- Ensures alignment between static and dynamic models.
- Should be performed continuously.
- Involves cross-functional team.

33.17 Scenario Walk-Through

- Validate scenarios using sequence diagrams.
- Ensure operations exist in class diagrams.
- Verify object interactions.

33.18 Event Tracing

- Ensure messages correspond to class operations.
- Validate associations between classes.
- Check events in state diagrams.

33.19 Documentation Review

- Ensure all classes are documented.
- Verify completeness of attributes and operations.
- Enforce modeling standards.

34 MCQs: 4+1 View Model

1. The 4+1 View Model was proposed by:
 - (a) Grady Booch
 - (b) Ivar Jacobson
 - (c) **Philippe Kruchten**
 - (d) James Rumbaugh
2. Which view is considered the central view in the 4+1 model?
 - (a) Logical View
 - (b) Deployment View
 - (c) Process View
 - (d) **Use Case View**
3. Which view addresses concurrency and synchronization?
 - (a) Logical View
 - (b) **Process View**
 - (c) Implementation View
 - (d) Deployment View
4. Component diagrams belong to which view?
 - (a) Logical View
 - (b) Process View
 - (c) **Implementation View**
 - (d) Use Case View
5. Which view is mainly used by system administrators?
 - (a) Logical View
 - (b) Process View
 - (c) **Deployment View**
 - (d) Use Case View
6. Homogenization mainly helps to:
 - (a) Increase redundancy
 - (b) **Remove inconsistencies**
 - (c) Add new classes
 - (d) Speed up coding

7. Which activity ensures messages correspond to class operations?
 - (a) Scenario Walk-through
 - (b) **Event Tracing**
 - (c) Documentation Review
 - (d) Class Splitting
8. Which view focuses on system functionality from the user perspective?
 - (a) Logical View
 - (b) **Use Case View**
 - (c) Process View
 - (d) Deployment View

35 MCQs: 4+1 View Model (Easy to Tricky)

Easy Level

1. The primary goal of the 4+1 View Model is to:
 - (a) Replace UML
 - (b) Reduce coding effort
 - (c) **Separate concerns of different stakeholders**
 - (d) Eliminate documentation
2. Which view captures system requirements in the form of scenarios?
 - (a) Logical View
 - (b) Process View
 - (c) Deployment View
 - (d) **Use Case View**
3. Which UML diagram is MOST commonly used in the Use Case View?
 - (a) Class Diagram
 - (b) Deployment Diagram
 - (c) **Use Case Diagram**
 - (d) Component Diagram
4. Logical View mainly represents:
 - (a) Hardware topology
 - (b) User interactions
 - (c) **Classes, objects, and relationships**
 - (d) File organization

Medium Level

5. Which diagram helps show object interactions over time?
 - (a) Class Diagram
 - (b) Component Diagram
 - (c) **Sequence Diagram**
 - (d) Deployment Diagram
6. The Process View mainly focuses on:
 - (a) Functional requirements
 - (b) Static class structure
 - (c) **Concurrency, threads, and synchronization**
 - (d) User interface design
7. Which view is responsible for mapping software artifacts to hardware?
 - (a) Logical View
 - (b) Implementation View
 - (c) **Deployment View**
 - (d) Use Case View
8. Which stakeholders benefit MOST from the Implementation View?
 - (a) Customers
 - (b) Testers
 - (c) **Developers**
 - (d) End users
9. Homogenization should be performed:
 - (a) Only at project start
 - (b) Only at project end
 - (c) **Continuously during development**
 - (d) Only during testing

Tricky / Conceptual Level

10. Why is the Use Case View called the "+1" in the 4+1 model?
 - (a) It is optional
 - (b) It replaces all other views
 - (c) **It drives and validates all other views**
 - (d) It is used only during testing
11. Which situation BEST justifies eliminating a control class?
 - (a) It has many attributes
 - (b) It has complex logic
 - (c) **It only passes data without sequencing logic**
 - (d) It is used by multiple actors
12. Scenario walk-throughs are primarily used to:
 - (a) Optimize performance
 - (b) Create new classes
 - (c) **Check consistency between static and dynamic models**
 - (d) Generate test cases automatically
13. Event tracing ensures consistency between:
 - (a) Use case and deployment diagrams
 - (b) Component and class diagrams
 - (c) **Sequence diagrams and class/state diagrams**
 - (d) Activity and deployment diagrams
14. If a system runs on a single processor, which view may be unnecessary?
 - (a) Logical View
 - (b) Implementation View
 - (c) **Deployment View**
 - (d) Use Case View
15. Which of the following is NOT a benefit of the 4+1 View Model?
 - (a) Better separation of concerns
 - (b) Improved stakeholder communication
 - (c) **Elimination of design complexity**
 - (d) Easier system understanding

36 MCQs: Implementation Diagrams

1. Component and Deployment diagrams are:
 - (a) Behavioral diagrams
 - (b) Dynamic diagrams
 - (c) **Structural diagrams**
 - (d) Interaction diagrams
2. A component is:
 - (a) A single class
 - (b) A data structure
 - (c) **An autonomous and reusable unit**
 - (d) A use case
3. Which notation represents a provided interface?
 - (a) Socket
 - (b) Dashed arrow
 - (c) **Ball (lollipop)**
 - (d) Port
4. Required interfaces represent:
 - (a) Services offered
 - (b) Internal behavior
 - (c) **Services needed by a component**
 - (d) Execution threads
5. Which connector links required and provided interfaces?
 - (a) Dependency connector
 - (b) Delegation connector
 - (c) **Assembly connector**
 - (d) Association
6. A port is used to:
 - (a) Store data
 - (b) Execute code
 - (c) **Define interaction points**
 - (d) Replace components

7. Black-box view of a component shows:
- (a) Internal classes
 - (b) Realization details
 - (c) **Interfaces only**
 - (d) Artifacts
8. Which diagram maps software to hardware?
- (a) Class diagram
 - (b) Component diagram
 - (c) **Deployment diagram**
 - (d) Sequence diagram
9. Nodes in deployment diagrams represent:
- (a) Classes
 - (b) Files
 - (c) **Hardware or execution environments**
 - (d) Interfaces
10. Artifacts are:
- (a) Logical models
 - (b) Behavioral elements
 - (c) **Physical files**
 - (d) Use cases

37 Deployment Diagrams

37.1 Introduction

- Deployment diagrams are part of the **Physical View** of a system.
- They show the physical elements (hardware and software) of a system and how software components are deployed onto hardware.
- Deployment diagrams illustrate the relationship between **nodes** (hardware) and **artifacts** (software), and the communication between them.

37.2 Hardware Elements (Nodes)

- Hardware nodes include:
 - Servers, Desktop PCs, Disk Drives
 - Embedded processors, IoT devices, Peripherals
- Execution nodes include:
 - Operating Systems
 - Application servers, Web servers, J2EE containers
- Nodes can be **nested** to represent hierarchy.

37.3 Connections

- Connections show communication paths between nodes.
- Example: TCP/IP connections between client and server.

37.4 Artifacts

- An **artifact** represents a physical piece of information (software):
 - Executable files, source code files, configuration files, database tables
- Notation: Rectangle with **artifact** keyword and optional document icon.

37.5 When to Use Deployment Diagrams

- Early stages: Identify general system configuration (hardware/software).
- Later stages: Show detailed architecture, technology stack, communication protocols, and artifacts.
- Example: A web application with a web server, application server, database, and clients accessing via browsers.

37.6 Example Elements in a Deployment Diagram

- Web Server Node
- Application Server Node
- Database Server Node
- Client Node (Desktop, Mobile)
- Software Artifacts: WebApp.exe, DatabaseTable.sql, ConfigFile.xml

37.7 MCQs: Deployment Diagrams

1. Deployment diagrams primarily show:
 - (a) Logical relationships between classes
 - (b) **Physical deployment of software on hardware**
 - (c) User interactions
 - (d) Sequence of operations
2. Which of the following is considered a hardware node?
 - (a) Operating System
 - (b) **Server**
 - (c) Source Code File
 - (d) Class Diagram
3. What does an artifact represent in a deployment diagram?
 - (a) A class or object
 - (b) **A physical piece of software like executable or database table**
 - (c) A communication path
 - (d) A user interface screen
4. Which connection type is commonly shown in deployment diagrams?
 - (a) Inheritance
 - (b) Aggregation
 - (c) **Communication path (e.g., TCP/IP)**
 - (d) Composition
5. Nodes in a deployment diagram can be:
 - (a) Only hardware nodes
 - (b) Only software artifacts
 - (c) **Nested to show hierarchy of hardware/software**
 - (d) Only clients
6. At which stage are deployment diagrams used?
 - (a) Only during implementation
 - (b) Only during analysis
 - (c) **Early for configuration, later for detailed architecture**
 - (d) Only for testing

38 Timing Diagrams

38.1 Introduction

A **Timing Diagram** shows the interaction of objects and the changes in their states along a time axis. It is useful to reason about active objects and their behavior over time.

- X-axis: time units
- Y-axis: objects and their states
- Shows state changes, events, messages, duration, and time constraints

38.2 Elements of Timing Diagram

- **Lifeline**: Represents an individual participant in the interaction
- **State or Condition Timeline**: Shows states of an object or attribute over time
- **Duration Constraint**: Interval of time that a state or action should occur
- **Time Constraint**: Specifies a time interval in which an event should occur
- **Destruction Occurrence**: Represents destruction of the lifeline, denoted by a cross (X)
- **Messages**: Communication between lifelines over time

38.3 Timing Diagram Notations

- **Concise Notation (Value Lifeline Diagram)**: States shown as areas
- **Robust Notation (State Lifeline Diagram)**: States shown as lines, with messages and events
- **Duration and Time Constraints**: Represented as vertical/horizontal intervals

38.4 Examples

- **Coffee Pot Example**: Pump on, hotplate on, duration constraints between them
- **Seminar Lifecycle**: Proposed → Scheduled → Enrolling → Being Taught → Final Exams → Closed
- **Website Latency**: User request → DNS resolution → Data retrieval → Response rendered
- **Door Control Example**: Sensor and processor interaction, verification within 4 seconds, door open for 10 seconds

38.5 4+1 View Model and Homogenization of Classes

- Timing diagrams help in understanding dynamic behavior for each view
- Homogenization ensures consistency in classes and interactions for predictable behavior

38.6 MCQs

1. What does the X-axis in a timing diagram represent?
 - (a) Objects
 - (b) **Time units**
 - (c) States
 - (d) Messages
2. What symbol is used to show destruction of a lifeline?
 - (a) Circle
 - (b) Arrow
 - (c) **Cross (X)**
 - (d) Rectangle
3. Which notation shows states as lines in timing diagrams?
 - (a) Concise Notation
 - (b) **Robust Notation**
 - (c) Value Lifeline Notation
 - (d) UML Class Notation
4. A duration constraint in a timing diagram specifies:
 - (a) When an event occurs
 - (b) **How long a state or action should occur**
 - (c) The object name
 - (d) The message type
5. Which element represents an individual participant in a timing diagram?
 - (a) Artifact
 - (b) **Lifeline**
 - (c) Duration
 - (d) Constraint

39 Entity, Control, and Boundary (ECB) Design Pattern

39.1 Overview

The ECB pattern is a refinement of the MVC (Model-View-Controller) design pattern. It decomposes the application into three types of objects:

- **Entity objects:** represent the core business data and logic (e.g., Student, Course, Insurance Policy).
- **Boundary objects:** handle communication between the system and external actors (e.g., Forms, Windows, Devices).
- **Control objects:** encapsulate application-specific business logic and coordinate interactions between boundary and entity objects.

39.2 Boundary Objects

- Interface between system and actors.
- Examples: user interface forms, system interfaces, device interfaces.
- Responsibilities: insulate system from external changes, coordinate actor interactions.

39.3 Control Objects

- Manage business logic and use case flow.
- Coordinate interactions between boundary and entity objects.
- Example: *Change Insurance Contract*, *Register Student*.

39.4 Entity Objects

- Represent persistent information and business objects.
- Example: *Student*, *Course*, *Insurance Policy*.
- Responsibilities: store, manage, and update data.

39.5 ECB Communication Rules

- Boundary objects interact with control objects.
- Control objects interact with both boundary and entity objects.
- Entity objects interact only with control objects.
- Direct communication between boundary and entity objects is not allowed.

39.6 Example: University Course Registration System

- **Boundary Objects:** CourseListForm, StudentForm, ConfirmationScreen.
- **Control Objects:** CourseRegistrationControl, AdvisorApprovalControl.
- **Entity Objects:** Student, Course, Registration.

39.7 Robustness Diagram

- Used to model ECB pattern in use case realization.
- Shows interactions between boundary, control, and entity objects.
- Example: Teacher clicks “Add courses” → Boundary class handles UI → Control class registers student → Entity class updates Student and Registration data.

39.8 MCQs: ECB Pattern

1. ECB pattern is a refinement of which design pattern?
 - (a) Singleton
 - (b) Observer
 - (c) **MVC**
 - (d) Factory
2. Which ECB object handles communication with actors?
 - (a) Entity
 - (b) **Boundary**
 - (c) Control
 - (d) Model
3. Which ECB object contains the business logic?
 - (a) Entity
 - (b) Boundary
 - (c) **Control**
 - (d) Interface
4. Which ECB object is persistent and stores system data?
 - (a) Boundary
 - (b) Control
 - (c) **Entity**

- (d) Adapter
- 5. Which communication is NOT allowed in ECB pattern?
 - (a) Boundary $\rightarrow$ Control
 - (b) Control $\rightarrow$ Entity
 - (c) **Boundary $\rightarrow$ Entity**
 - (d) Control $\rightarrow$ Boundary
- 6. In a course registration system, “CourseRegistrationControl” is an example of:
 - (a) Boundary object
 - (b) **Control object**
 - (c) Entity object
 - (d) View object
- 7. “Student” class that stores data about the student is an example of:
 - (a) Control
 - (b) Boundary
 - (c) **Entity**
 - (d) Adapter
- 8. “ConfirmationScreen” in the registration system is an example of:
 - (a) Entity
 - (b) **Boundary**
 - (c) Control
 - (d) Model

40 Use Case Realization and Analysis Classes

40.1 Use Case Realization

- Describes how a particular use case is realized in the design model.
- Focuses on the collaboration of objects to achieve the use case.
- Can be modeled using:
 - **Interaction diagrams** (sequence or collaboration diagrams): Show detailed collaborations.
 - **Class diagrams**: Show the classes participating and their relationships.

40.2 Responsibilities

- A **designer** is responsible for the integrity of use-case realization.
- Coordinates with class designers to understand class roles.
- Helps determine and refine class responsibilities and interfaces.

40.3 Analysis Classes - Early Conceptual Model

- Represent an early conceptual model of the system.
- Captures required behaviors and relationships of the system.
- Fluid and evolving; may morph into subsystems, components, or other design elements.
- Provide a rough-cut view of the system's object model.

40.4 Identifying Candidate Classes from Behavior

- Use three perspectives to identify classes:
 1. **Boundary**: Interfaces between system and actors.
 2. **Entity**: Information and core system objects.
 3. **Control**: Logic managing flow and interactions.
- Stereotypes (boundary, control, entity) are used during analysis.
- Distributing use-case behavior among these classes ensures a robust design.

40.5 Analysis Classes: A First Step Towards Executables

- Represent early objects with responsibilities and behaviors.
- Capture primary functional and interface requirements.
- Provide the basis for further design and implementation.

40.6 Entity, Control, and Boundary (ECB) Design Pattern

- Refinement of the MVC (Model-View-Controller) pattern.
- Decomposes the system into:
 - **Entity Objects**: Represent business data and logic.
 - **Control Objects**: Encapsulate business logic and coordinate interactions.
 - **Boundary Objects**: Interface with actors and external systems.
- Rules:

- Boundary objects communicate with control objects.
- Control objects communicate with both boundary and entity objects.
- Entity objects communicate only with control objects.
- Direct boundary-to-entity communication is not allowed.

40.7 Example: University Course Registration System

- **Boundary Objects:** CourseListForm, StudentForm, ConfirmationScreen
- **Control Objects:** CourseRegistrationControl, AdvisorApprovalControl
- **Entity Objects:** Student, Course, Registration

40.8 MCQs

1. Use-case realization describes:
 - (a) Hardware deployment
 - (b) System configuration
 - (c) **How a use case is implemented with collaborating objects**
 - (d) Programming language syntax
2. Interaction diagrams in use-case realization show:
 - (a) Class attributes
 - (b) Hardware nodes
 - (c) **Object collaborations over time**
 - (d) Database schemas
3. Analysis classes represent:
 - (a) Final implementation classes
 - (b) **Early conceptual model of the system**
 - (c) Hardware configuration
 - (d) Network topology
4. Candidate classes can be identified from:
 - (a) Actors only
 - (b) Database tables only
 - (c) **Boundary, Entity, and Control perspectives**
 - (d) Network components
5. In ECB pattern, which object stores business data?

- (a) Boundary
 - (b) Control
 - (c) **Entity**
 - (d) Interface
6. Boundary objects are responsible for:
- (a) Core business logic
 - (b) Database transactions
 - (c) **Interfacing with actors and external systems**
 - (d) System monitoring
7. Control objects are responsible for:
- (a) Rendering UI only
 - (b) Storing persistent data
 - (c) **Managing business logic and coordinating interactions**
 - (d) Configuring hardware
8. Which communication is NOT allowed in ECB?
- (a) Boundary $\rightarrow$ Control
 - (b) Control $\rightarrow$ Entity
 - (c) **Boundary $\rightarrow$ Entity**
 - (d) Control $\rightarrow$ Boundary

40.9 MCQs on Use-Case Realization

1. Use-case realization is primarily concerned with:
- (a) Database design
 - (b) **Object interactions to implement a use case**
 - (c) Hardware configuration
 - (d) Programming language choice
2. Which diagram is NOT typically used for use-case realization?
- (a) Sequence diagram
 - (b) Collaboration diagram
 - (c) Class diagram
 - (d) **Deployment diagram**

3. Who is responsible for creating use-case realizations?
 - (a) Network engineer
 - (b) **Designer/Analyst**
 - (c) Database administrator
 - (d) End-user
4. Which classes are commonly involved in a use-case realization?
 - (a) **Boundary, Control, Entity**
 - (b) Boundary, Control, Entity
 - (c) Interface, Database, Utility
 - (d) Actor, Interface, Node
5. The main benefit of use-case realization is:
 - (a) Faster programming
 - (b) **Ensuring system behavior matches requirements**
 - (c) Reduced network latency
 - (d) Enhanced security

41 Use Cases

41.1 Overview

- A **Use Case** represents a scenario-based technique in UML.
- It describes system functionality from the user's perspective.
- Shows interactions between actors (users or other systems) and the system.

41.2 Use Case Diagram Components

- **Actors:** Entities that interact with the system (human, system, device).
- **Use Cases:** Functionalities the system provides.
- **Relationships:** Connections between actors and use cases.
- **Boundary:** System boundary rectangle showing system limits.

41.3 Types of Actors

- **Primary Actors:** Initiate actions and have goals fulfilled by system (e.g., Cashier).
- **Secondary Actors:** Provide services to the system (e.g., Payment service).
- **Offstage Actors:** Interested in outcomes but not active (e.g., Government Tax Agency).

41.4 Relationships in Use Cases

- **Association:** Simple communication between actor and use case.
- **Generalization:**
 - Actor generalization: Child actor inherits role/properties of parent actor.
 - Use case generalization: Child use case inherits behavior of parent use case.
- **Include:** Adds common behavior from another use case. Notation: <<include>>.
- **Extend:** Adds optional behavior to a base use case. Notation: <<extend>>.

41.5 Types of Use Cases

- **Brief:** One-paragraph summary of main success scenario.
- **Casual:** Informal multiple-paragraph description covering variations.
- **Fully Dressed:** Detailed steps, preconditions, postconditions, alternate flows.

41.6 Writing a Use Case

- **Name:** Verb + noun, e.g., “Register New User”.
- **Brief Description:** Overview of the scenario.
- **Actors:** Primary and secondary actors involved.
- **Preconditions:** Conditions true before scenario starts.
- **Basic Flow:** Step-by-step interaction between actors and system.
- **Alternate Flows:** Optional variations or paths.
- **Exception Flows:** Errors or abnormal cases preventing goal achievement.
- **Post Conditions:** Conditions true after scenario completion.

41.7 Example Use Case: Purchase at Checkout

- **Actors:** Customer (Primary), Cashier (Primary)
- **Brief Description:** Customer purchases items with cash. Cashier records items, accepts payment, and customer leaves with items.
- **Basic Flow:**
 1. Customer selects items.
 2. Cashier records items in POS system.

3. Customer pays cash.
4. POS system updates inventory and prints receipt.
5. Customer leaves.

41.8 Exercises

- Draw a use case diagram for a Hospital Reception System.
- Draw a use case diagram for a Vehicle Sales System.
- Identify functional requirements from a university course registration scenario.

41.9 MCQs

1. A use case primarily describes:
 - (a) Hardware configuration
 - (b) **System functionality from the user's perspective**
 - (c) Database tables
 - (d) Programming language syntax
2. Which is NOT a type of use case?
 - (a) Brief
 - (b) Casual
 - (c) Fully Dressed
 - (d) **Technical Use Case**
3. The <<include>> relationship is used to:
 - (a) Replace the base use case
 - (b) Add optional behavior
 - (c) **Add common behavior from another use case**
 - (d) Inherit behavior from actor
4. Primary actors are:
 - (a) Interested parties not initiating actions
 - (b) **Actors who initiate actions to achieve goals**
 - (c) External systems only
 - (d) Offstage stakeholders
5. Preconditions in a use case:

- (a) Are optional system states
 - (b) **Must be true before the use case starts**
 - (c) Are always checked by the actor
 - (d) Are the same as postconditions
6. An <<extend>> relationship indicates:
- (a) Mandatory behavior
 - (b) Optional behavior
 - (c) **Optional behavior extending a base use case**
 - (d) Actor inheritance
7. A fully dressed use case includes:
- (a) Only a brief paragraph
 - (b) Only the main success scenario
 - (c) **Detailed steps, alternate flows, preconditions, and postconditions**
 - (d) Only the actors involved